

Machine Learning III

Digitization and Data Analytics — Complete Study Guide

Dimension Reduction (PCA, t-SNE, Autoencoders) · Linear Regression ·
Logistic Regression

Detailed notes, diagrams, worked examples, exam-style Q&A and
research-oriented discussion questions.

Based on: *Iryna Okhrin — Digitization and Data Analytics, Lecture 7*
Compiled study guide — not an official TU Dresden exam paper

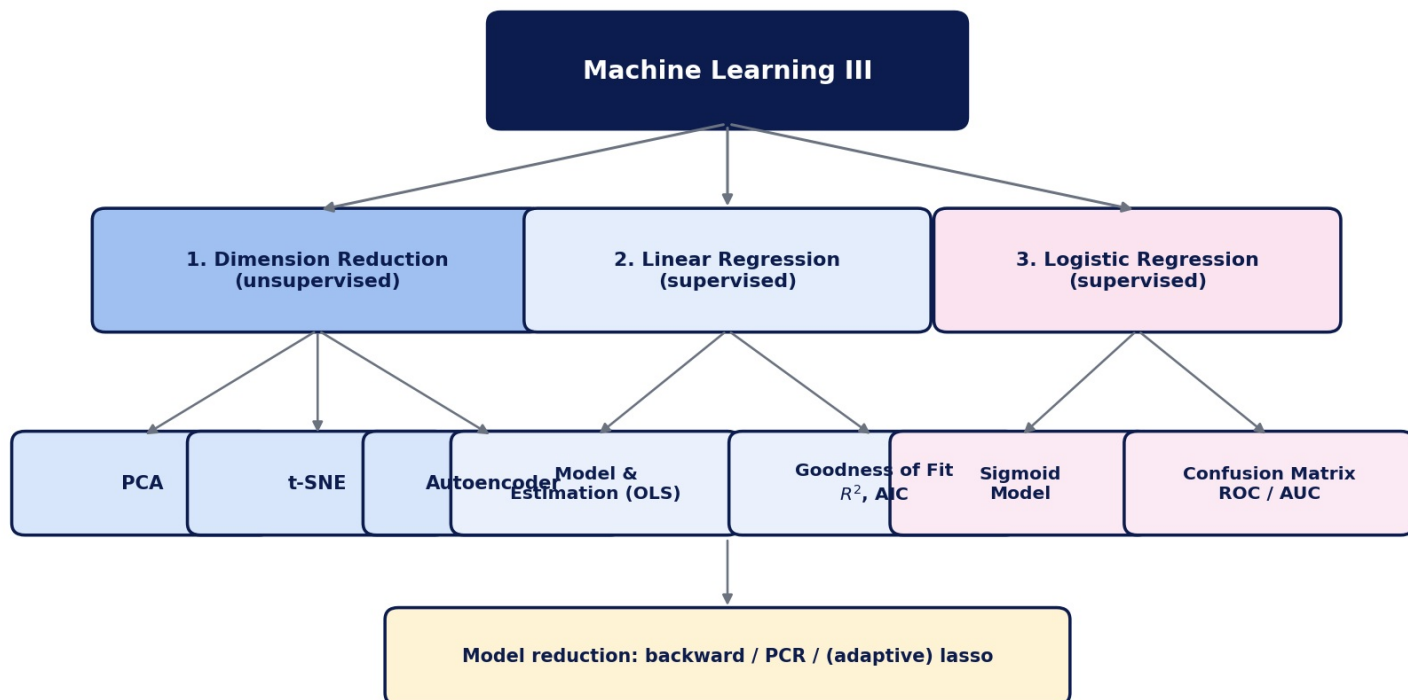
Contents

1. Concept Map & Big Picture	3
2. Dimension Reduction — Motivation	4
3. PCA — Principal Component Analysis	5
4. t-SNE	9
5. Autoencoders	11
6. Linear Regression	12
7. Logistic Regression	16
8. Exam-Style Question Bank (40+ Q&A)	19
9. Research-Oriented Questions	27
10. One-Page Cheat Sheet	30

Page numbers are approximate and depend on your PDF viewer's rendering.

1. Concept Map & Big Picture

This lecture (Lecture 7 of 12, "Machine Learning III") sits between classical unsupervised/supervised methods (clustering, decision trees, Bayes classifiers — Lecture 6) and Deep Learning (Lecture 8-9). It covers **three families of models**: two are about compressing data (dimension reduction), and two are regression models for prediction/explanation.



Why this lecture matters for the exam: it is the bridge between "explain the data with fewer numbers" (PCA/t-SNE/Autoencoder) and "explain/predict a target variable from features" (linear & logistic regression). Expect questions that ask you to *connect* these — e.g. use PCA to reduce collinearity before regression (PCR), or explain why regression assumptions fail when features are highly correlated.

2. Dimension Reduction — Motivation

Aim: transform data X to data Y of lower dimension without losing (much) information, i.e. find a function $f: X \rightarrow Y$ that is optimal in some sense.

Why reduce dimension?

- **Visualization** — humans can only really interpret 2D/3D plots (e.g., 10D $\rightarrow$ 2D).
- **Noise reduction** — outliers/mistakes often live in a single, unimportant dimension; dropping it removes noise.
- **Computational efficiency** — e.g. inverting a 2×2 matrix is far cheaper than a 5×5 one (relevant for OLS: $(X'X)^{-1}$).
- **Storage memory** — fewer numbers to store, especially at scale (images, sensor streams).
- **Improved accuracy** — removing redundant/collinear variables can reduce variance of estimators (bias-variance trade-off).

Running example: students' grades

50 students $\times$ 7 subjects (math, algebra, programming, statistics, English, philosophy, history) $\rightarrow$ a 50×7 matrix X . Idea: a single latent factor (e.g., "IQ") could explain grades in *all* subjects:

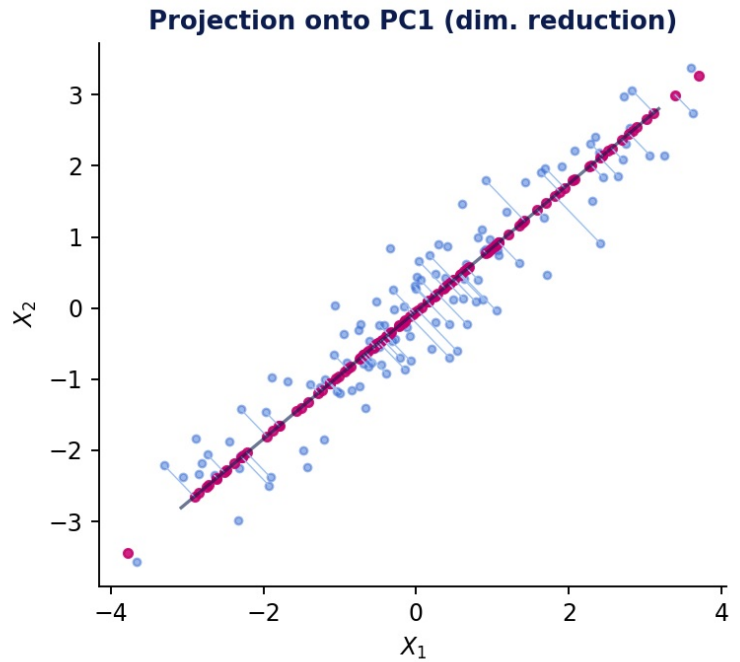
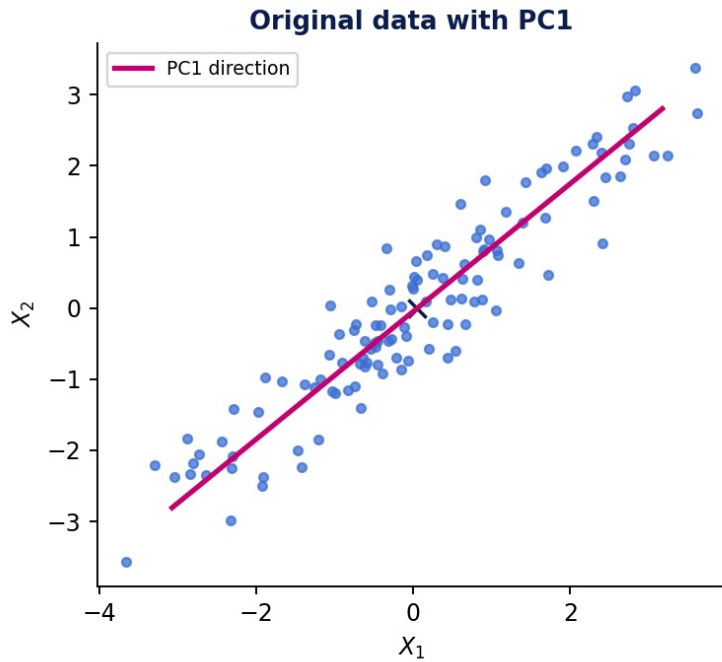
$$grade_{ij} \approx a_j + b_j \cdot IQ_i$$

If this holds, all seven subjects are linearly correlated and the 50×7 data can be represented (almost) by a single 50×1 vector of latent IQ scores. This is exactly the intuition PCA formalizes.

3. PCA — Principal Component Analysis

3.1 Idea

Find direction β ($\|\beta\| = 1$) such that $Y = \beta X$ has **maximum variance**: $\sum \text{var}(Y_j) \rightarrow \max$.



PCA looks for the linear transformation (rotation) of the data that concentrates as much variance as possible into as few new coordinates as possible. The **slope** of the projection direction is what matters (it determines how much variance is captured); the **shift** (mean) is irrelevant to the direction-finding problem — data is always mean-centered first.

3.2 Eigenvalues and Eigenvectors

Empirical covariance matrix: $\text{cov}(X) = (1/n)(X - \bar{X})(X - \bar{X})'$

Every covariance matrix admits a spectral decomposition:

$$\text{cov}(X) = \Gamma \Lambda \Gamma'$$

- $\Lambda = \text{diag}(\lambda_1, \dots, \lambda_p)$ — eigenvalues, ordered largest to smallest.
- $\Gamma = (\gamma_1, \dots, \gamma_p)$ — matrix of eigenvectors (the principal axes).
- $\text{rank}(\text{cov}(X)) = \#\{\lambda_i \neq 0\}$
- $\sum \text{var}(X_i) = \sum \lambda_i$ (total variance is conserved, just redistributed).
- Eigenvectors have unit length ($\gamma_i' \gamma_i = 1$) and are orthogonal ($\gamma_i' \gamma_j = 0, i \neq j$).

3.3 Principal Components (PCs)

$$Y = \Gamma(X - \bar{X}) \quad \text{i.e.} \quad Y_j = \gamma_j'(X - \bar{X})$$

- PCs have zero mean: $\bar{Y} = 0$.
- $\text{cov}(Y) = \Lambda$ — the PCs are **uncorrelated** by construction, with variances $\lambda_1 \geq \dots \geq \lambda_p$.
- Dimension reduction = keep only the first $q < p$ PCs, which capture most of the variance.

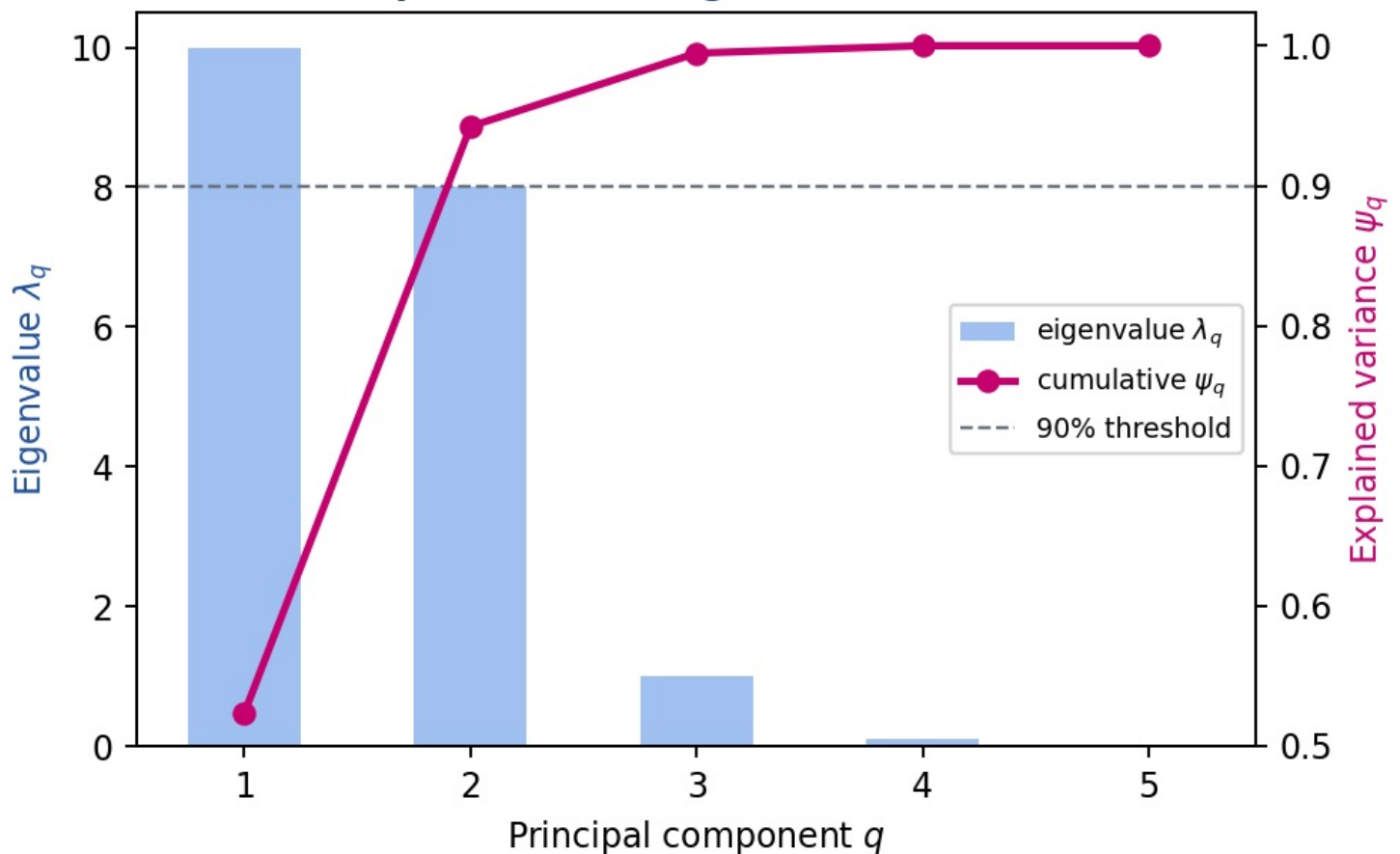
3.4 How Many PCs to Keep?

$$\psi_q = (\lambda_1 + \dots + \lambda_q) / (\lambda_1 + \dots + \lambda_p), \quad q \in \{1, \dots, p\}$$

$\psi_q \in (0, 1]$, increases with q , $\psi_p = 1$. Two common decision rules:

1. **Explained-variance threshold** — keep enough PCs so $\psi_q \geq 90\%$ or 95% .
2. **Elbow method** — plot λ_q (scree plot) and cut where the curve "bends" / flattens.

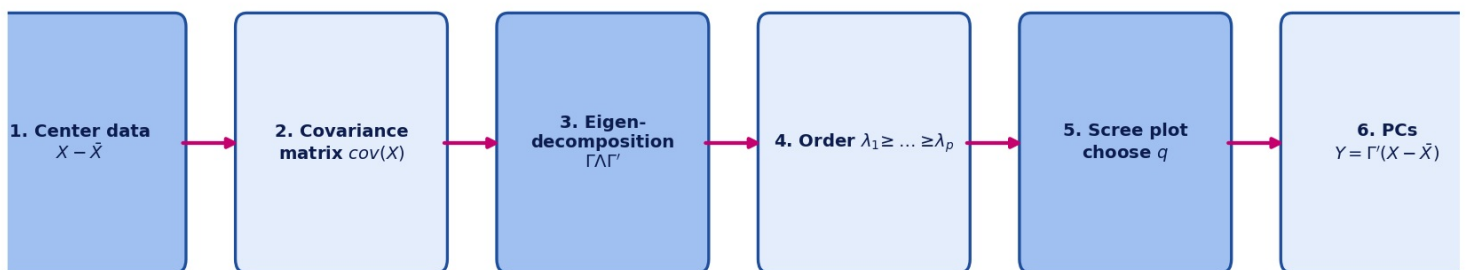
Scree plot — choosing the number of PCs



q	λ_q	Ψ_q
1	10	0.52
2	8	0.94
3	1	0.99
4	0.1	1.00
5	0	1.00

Worked example from the lecture: with 90–95% threshold, $q=2$ is already a very good choice (94% variance explained, elbow visible after PC2).

3.5 PCA Algorithm (end-to-end)



1. Compute covariance matrix $\text{cov}(X) = (X - \mu)'(X - \mu)$.
2. Eigen-decompose: get $\lambda = (\lambda_1, \dots, \lambda_p)$ and $\Gamma = (\gamma_1, \dots, \gamma_p)$.
3. Compute PCs: $Y = \Gamma'(X - \bar{X})$.
4. Use the scree plot to choose effective dimension q .
5. Visualize: raw data, scree plot, PCs, biplot.
6. Keep only the first q PCs for downstream analysis.

3.6 Visualization Tools

- **Data itself** — pairwise scatterplots of raw variables.
- **Scree plot** — eigenvalues / cumulative variance vs. q .
- **PC scatterplot** — the transformed, reduced data (e.g. PC1 vs PC2).
- **Biplot** — correlation $\rho_{ij} = \text{cor}(X_i, Y_j)$ shows how well PC j explains original variable X_i . Since $\sum_{j=1}^p \rho_{ij}^2 = 1$, we know $\sum_{j=1}^q \rho_{ij}^2 \leq 1$ — the biplot shows how close each variable's arrow is to the unit circle (close = well explained by the first q PCs).

3.7 PCA Summary

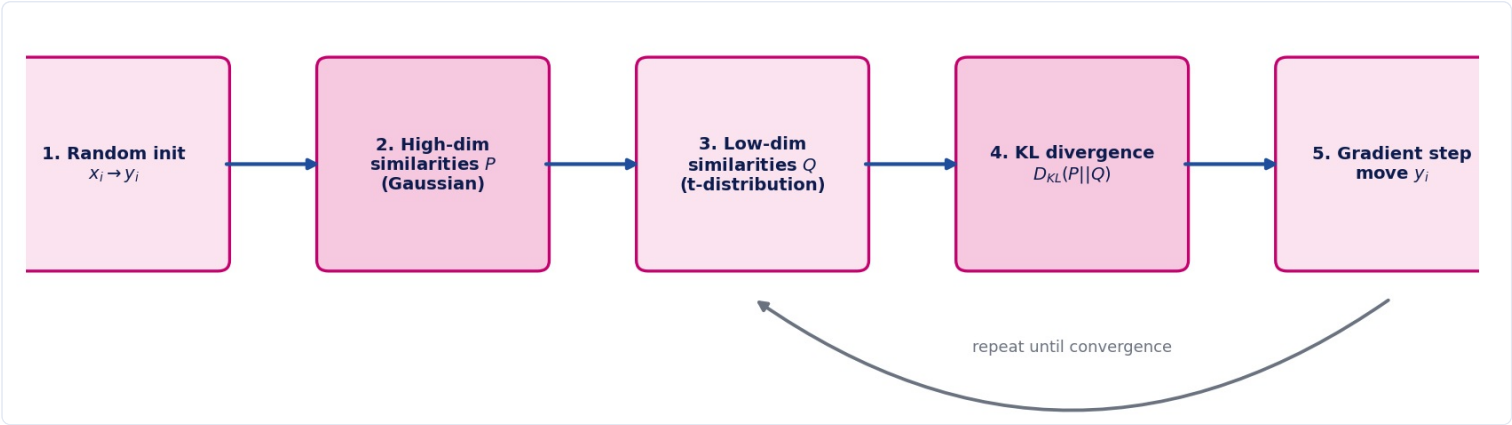
Advantages	Disadvantages
Easy, closed-form calculation (eigen-decomposition)	Only captures linear dependence (non-linear structure needs kernel PCA / autoencoders)
Easy to interpret (variance explained, loadings)	Sensitive to outliers (covariance is not robust)
Asymptotic convergence guarantees	Only for continuous variables

4. t-SNE — t-distributed Stochastic Neighbor Embedding

Aim: dimension reduction such that points that are **close in the original space stay close** in the low-dimensional embedding (a *local, non-linear* notion of similarity, unlike PCA's global linear variance criterion).

Example: a 64×64 RGB image is a single row of 64×64×3 = 12,288 numbers — far too high-dimensional to plot directly; t-SNE can embed such rows into 2D for visual inspection of clusters.

4.1 Algorithm



- Random initialization:** drop the points into a low dimension (usually 2 or 3), $x_i \rightarrow y_i$.
- High-dimensional similarities P:** $\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)$, then scaled/averaged into a proper probability distribution.
- Low-dimensional similarities Q:** $(1 + \|y_i - y_j\|^2)^{-1}$ (a Student-t / Cauchy kernel — heavier tails than Gaussian, which helps avoid the "crowding problem").
- Compare distributions** via Kullback-Leibler divergence: $D_{KL}(P||Q) = \sum_{i \neq j} p_{ij} \log(p_{ij}/q_{ij})$.
- Optimize:** move the y_i (gradient descent) to minimize D_{KL} , then recompute Q and repeat.

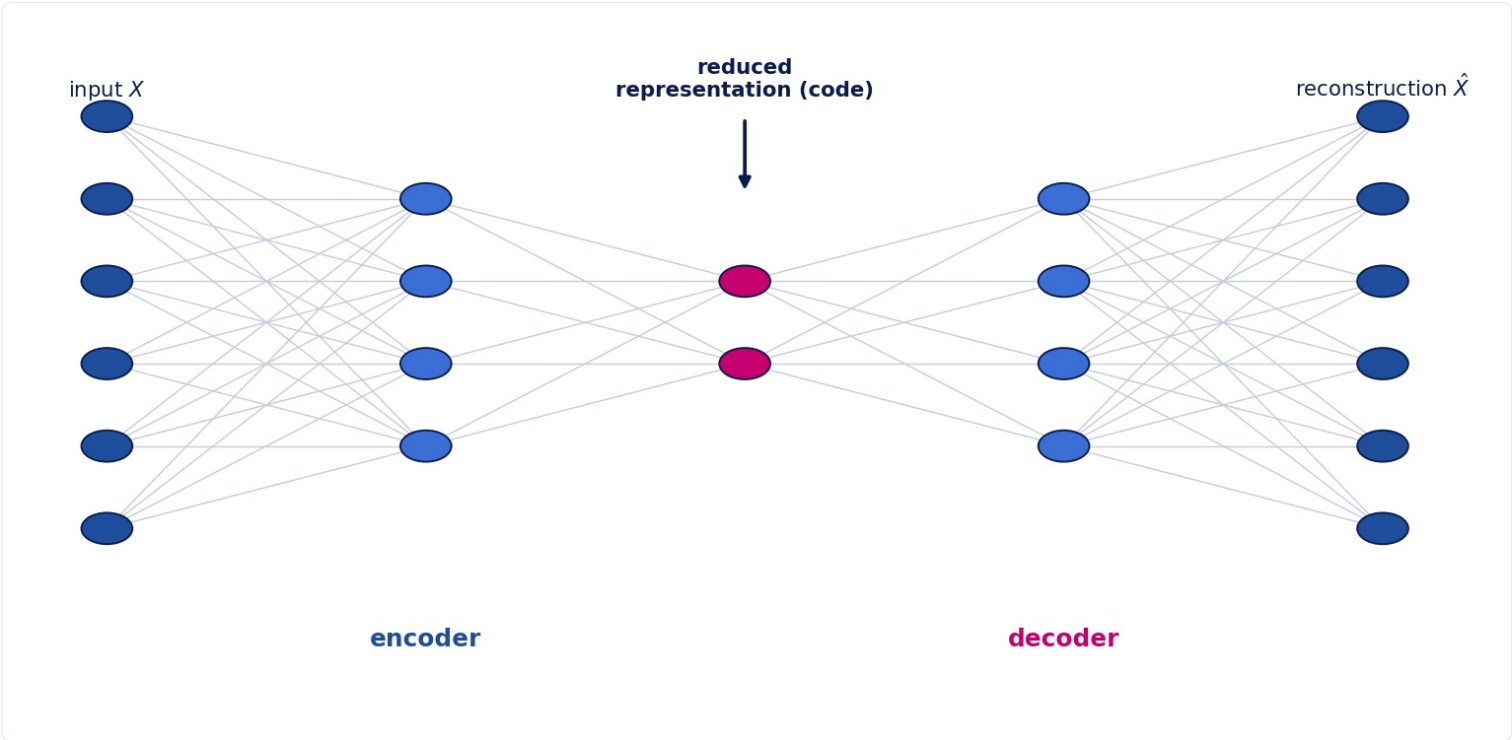
4.2 PCA vs. t-SNE

Aspect	PCA	t-SNE
Type of structure captured	Linear (global variance)	Any dependency (local neighborhoods)
Determinism	Deterministic, unique solution	Stochastic — different runs give different embeddings
Speed	Fast, closed-form (eigen-decomposition)	Computationally intensive, iterative optimization
New / out-of-sample points	Straightforward (just project: $Y = \Gamma'(x - \bar{X})$)	Not directly applicable to new points
Data type	Continuous only	Continuous only
Used for clustering?	Sometimes, as pre-processing	Yes, common for visual cluster discovery

Iris example insight from the lecture: PCA already separates the three species very well because the iris features are *strongly linearly correlated* — so the extra flexibility of t-SNE brings little benefit here, while costing determinism and speed.

5. Autoencoders

Idea: train a neural network to compress the data down (encoder) and then reconstruct it back to (nearly) the original (decoder). If reconstruction succeeds, the narrow middle layer ("bottleneck" / "code") must contain (almost) all the information needed to reconstruct X — i.e. it is a non-linear, learned dimension reduction.



Advantages	Disadvantages
Works for any data scale (not just continuous linear structure)	Computationally intensive to train
Applicable out-of-sample (just run new data through the trained encoder)	Only generalizes well to data similar to the training distribution

Connecting the three methods: PCA is actually the special case of a *linear* autoencoder with no activation function trained with squared-error loss — the bottleneck of such a linear autoencoder spans the same subspace as the first q principal components. This is a classic exam/interview connection question.

6. Linear Regression

Aim of any regression: model the relationship between a target Y and features X : $Y \approx m(X)$. The task can be prediction, explanation, or control. "Linear" means m is linear in the parameters: $y_i \approx \beta_0 + \beta_1 x_{i,1} + \dots + \beta_p x_{i,p}$, $i=1, \dots, n$.

6.1 Model

$$y_i = \beta_0 + \beta_1 x_{i,1} + \dots + \beta_p x_{i,p} + \varepsilon_i, \quad i=1, \dots, n \quad \Leftrightarrow \quad Y = X\beta + \varepsilon$$

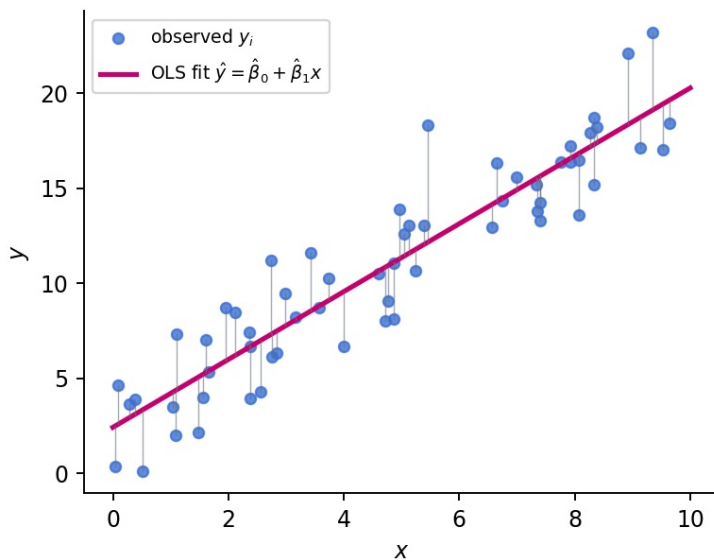
- Parameters $\beta = (\beta_0, \beta_1, \dots, \beta_p)'$ — to be estimated.
- Errors $\varepsilon = (\varepsilon_1, \dots, \varepsilon_n)'$ should be as small as possible, with $E(\varepsilon)=0$ and $\text{Var}(\varepsilon)=\sigma^2 I$ (i.i.d.).
- If $\varepsilon \sim N(0, \sigma^2 I)$ then $y \sim N(X\beta, \sigma^2 I)$, and a new prediction $y_0 \sim N(x_0'\beta, \sigma^2)$.

6.2 Estimation (Ordinary Least Squares)

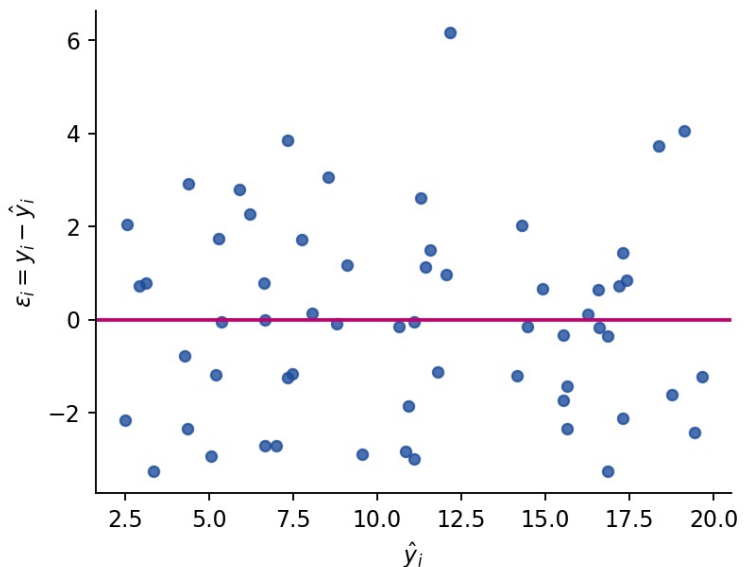
$$\|y - X\beta\|^2 \rightarrow \min_{\beta} \Rightarrow \hat{\beta} = (X'X)^{-1}X'Y$$

- Prediction at a new point x_0 : $\hat{Y}_0 = x_0'\hat{\beta} = x_0'(X'X)^{-1}X'Y$.
- $\text{Cov}(\hat{\beta}) = \sigma^2(X'X)^{-1}$; $\text{Cov}(\hat{Y}_0) = \sigma^2\{1 + x_0'(X'X)^{-1}x_0\}$.
- Problem:** $(X'X)^{-1}$ can be numerically unstable/undefined when X is very large (many columns) or when columns of X are linearly dependent (multicollinearity) — this is exactly where PCA/PCR or lasso-based model reduction becomes useful.

OLS fit with residuals



Residuals vs. fitted values



6.3 Goodness of Fit

Parameter significance

Under $\varepsilon \sim N(0, \sigma^2 I)$: test $H_{0j}: \beta_j=0$. If the p-value is small (e.g. ≤ 0.05), the coefficient β_j is statistically significant.

Coefficient of determination

$$R^2 = 1 - SS_{\text{residual}} / SS_{\text{total}} = 1 - \sum (y_i - \hat{y}_i)^2 / \sum (y_i - \bar{y})^2$$

$R^2 \in [0, 1]$; closer to 1 = better fit. Rough interpretive bands (as given in the lecture): 0–0.10 very weak, 0.10–0.30 weak, 0.30–0.60 moderate, 0.60–0.80 good, 0.80–1.00 very strong.

Dimension reduction of the model itself

R^2 can only stay the same or increase as you add variables (even irrelevant ones), so plain R^2 cannot be used to compare models of different size. Instead use:

$$\text{adjusted } R^2 = 1 - [SS_{\text{res}}/(n-k)] / [SS_{\text{tot}}/(n-1)] \rightarrow \max \quad \text{or} \quad \text{AIC} = \ln(\hat{\sigma}^2(k)) + k(2/n) \rightarrow \min$$

with $k \leq p$ the number of variables in the reduced model $\Rightarrow$ this underlies **backward variable selection**.

6.4 Variable Transformation

- **Dummies:** a categorical variable with d categories needs d−1 dummy variables.
- **Transform X, not Y:** any transformation of the predictors is fine, e.g. $y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 \log(x_{i1}) + \beta_3 x_{i1}^2 + \beta_4 x_{i1} x_{i2} + \dots + \epsilon_i$.
- **Avoid transforming Y** (e.g. $\log(y_i) = \beta_0 + \beta_1 x_{i1} + \dots$): this changes the error structure/interpretation and the comparability of R²/AIC across models, and complicates predictions (need back-transformation with bias correction).

6.5 Model Reduction Techniques

- **Backward elimination based on AIC** — drop the least useful variable repeatedly; simple but requires fitting many sub-models (worst case p(p+1)/2 fits).
- **PCR (Principal Component Regression)** — run PCA on X, replace X by the first important PCs, then run OLS on the PCs. Downside: you must still measure/observe every original variable; interpretability of coefficients is lost (they're in PC-space, not original-variable space).
- **(Adaptive) LASSO** (recommended in the lecture) — penalized regression:

OLS: $\|y - X\beta\|^2 \rightarrow \min_{\beta}$ *Lasso:* $\|y - X\beta\|^2 + \lambda \|\beta\|_1 \rightarrow \min_{\beta}$

λ is a hyperparameter (tuned e.g. via cross-validation); the L1 penalty shrinks some coefficients exactly to zero, giving automatic variable selection. Faster and typically achieves a sparser model than backward selection; the "adaptive lasso" variant reweights the penalty per coefficient for improved selection consistency.

6.6 Worked Example — Iris (Petal Length as target)

$Pe.Le = \beta_0 + \beta_1 Se.Le + \beta_2 Se.Wi + \beta_3 Pe.Wi + \beta_4 (Sp.=versi) + \beta_5 (Sp.=virgi) + \epsilon$

Term	$\hat{\beta}$	p-value
(Intercept)	−1.11	0.00
Sepal.Length	0.61	0.00
Sepal.Width	−0.18	0.03
Petal.Width	0.60	0.00
Species=versicolor	1.46	0.00
Species=virginica	1.97	0.00

R² = 0.9786, adjusted R² = 0.9778 — an excellent fit. Interpretation: Petal.Length increases with Sepal.Length and Petal.Width but decreases with Sepal.Width; virginica has the largest Petal.Length among species (largest dummy coefficient); all p-values < 0.05 ⇒ all terms significant ⇒ no further reduction needed (backward selection with AIC confirms: the "full" model has the best/lowest AIC of −395).

6.7 Linear Regression Summary

Algorithm	Advantages	Disadvantages
1. Prepare (visualize, decide transformations) 2. Estimate (OLS) 3. Interpret β , R ² 4. Visualize results (fit, residuals) 5. Check assumptions (normality, homogeneity, no strong collinearity) 6. Decide on model reduction	Easy closed-form calculation Easy to interpret Asymptotic convergence	Only linear dependencies Sensitive to outliers Continuous variables (or dummies) only

7. Logistic Regression

Same regression aim ($Y \approx m(X)$) but now **Y is categorical**; the model outputs class **probabilities**. Binary case = logistic regression; more than 2 classes = **multinomial** regression (its extension).

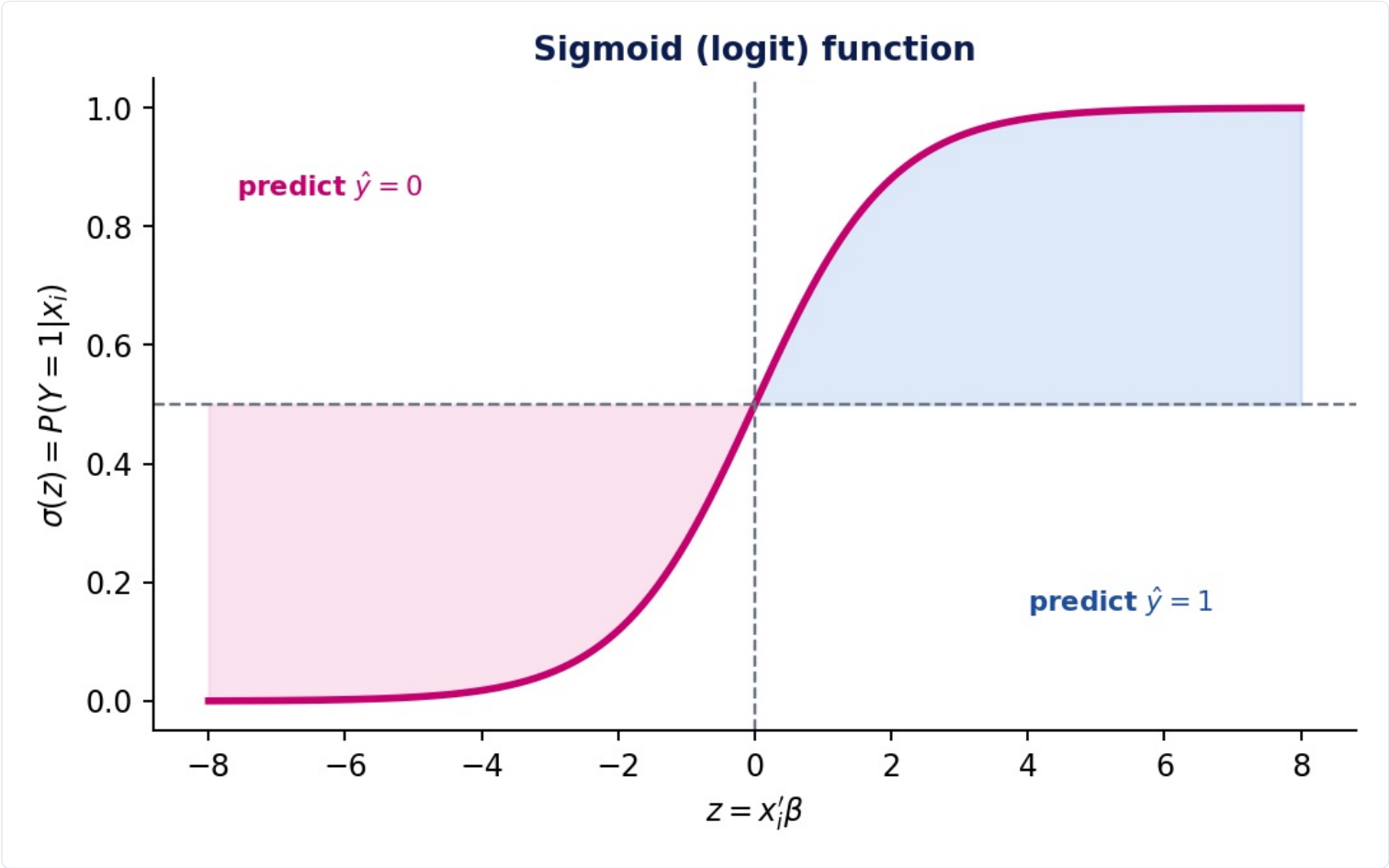
Iris example: logistic regression could model $P(\text{Species}=\text{'versicolor'}) \approx f(\text{Se.Le, Se.Wi, Pe.Le, Pe.Wi})$; multinomial regression would model $P(\text{Species}) \approx f(\dots)$ directly over all 3 classes at once.

7.1 Model

Let $Y \in \{0,1\}$. Aim: for a given x_i , estimate $P(Y=1|x_i)$ and $P(Y=0|x_i)$.

$$\log[P(Y=1|x_i) / P(Y=0|x_i)] = x_i'\beta \Leftrightarrow P(Y=1|x_i) = 1 / (1+\exp(-x_i'\beta)) = \sigma(z), \quad z=x_i'\beta$$

The left-hand side is the **log-odds (logit)**; $\sigma(\cdot)$ is the **sigmoid** function.



- If $\beta_i > 0$ then $P(Y=1)$ increases with x_i (and vice versa for $\beta_i < 0$).
- Prediction rule: $\hat{y}_0 = 1$ if $\hat{P}(Y=1) > 1/2$, else 0 (threshold is tunable, default 0.5).

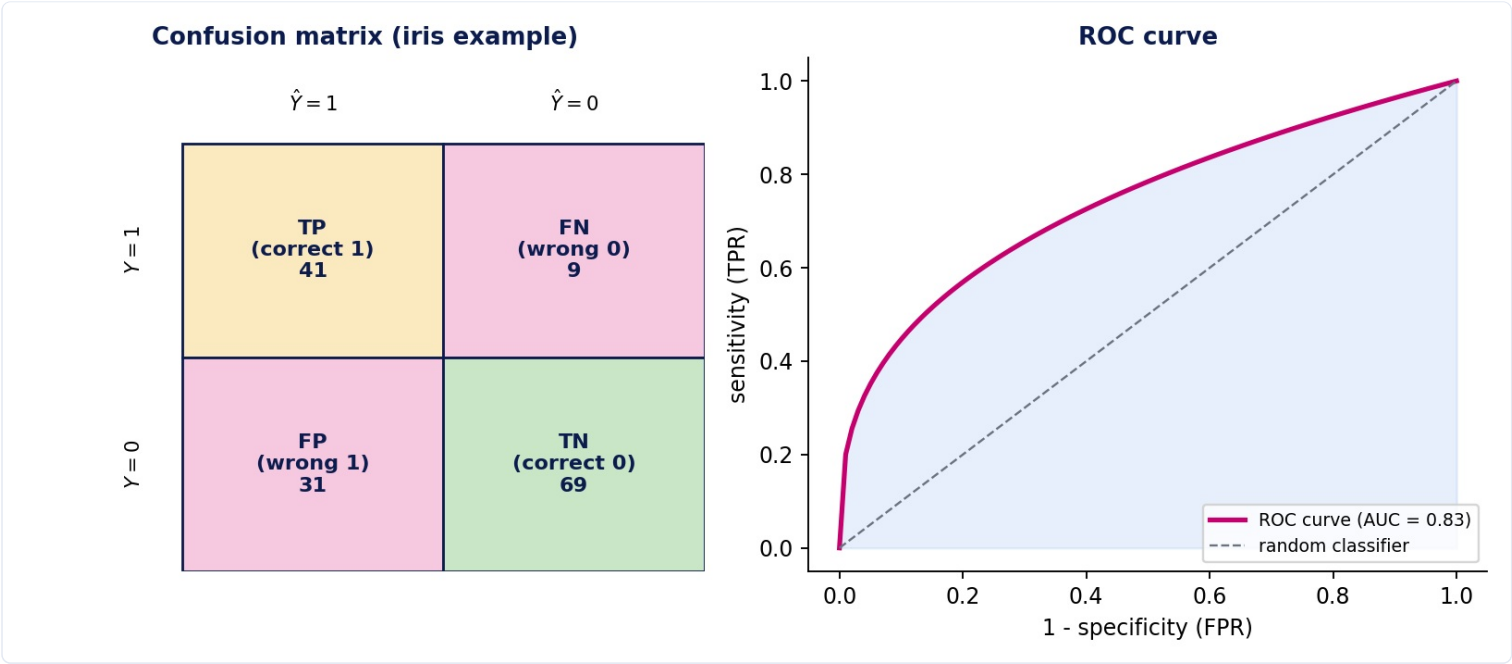
7.2 Worked Example — Iris (Species = versicolor)

$$P(\text{Sp.}=\text{versicolor}) = 1 / (1+\exp\{- (\beta_0+\beta_1\text{Se.Le}+\beta_2\text{Se.Wi}+\beta_3\text{Pe.Wi})\})$$

Term	$\hat{\beta}$	p-value
(Intercept)	7.38	0.00
Sepal.Length	-0.25	0.71
Sepal.Width	-2.80	0.00
Petal.Length	1.31	0.05
Petal.Width	-2.78	0.02

- $\beta_{\text{Pe.Le}} > 0 \Rightarrow P(\text{versicolor})$ increases with petal length.
- $\beta_{\text{Se.Le}}, \beta_{\text{Se.Wi}}, \beta_{\text{Pe.Wi}} < 0 \Rightarrow$ probability decreases with those three features.
- $|\beta_{\text{Se.Wi}}|$ is the largest $\Rightarrow P(\text{versicolor})$ changes fastest with sepal width.
- Se.Le has $p = 0.71 (> 0.05) \Rightarrow$ **not significant** $\Rightarrow$ candidate for removal via model reduction (same techniques as linear regression:

7.3 Goodness of Fit — Confusion Matrix, Sensitivity, Specificity, ROC/AUC



$accuracy = (TP+TN)/n$ $sensitivity = TP/(TP+FN)$ $specificity = TN/(TN+FP)$

Threshold	Accuracy	Sensitivity	Specificity
0.5 (default)	0.74	0.50	0.86
lowered (more liberal)	0.73	0.82	0.69

Lowering the classification threshold trades specificity for sensitivity (classic precision/recall-style trade-off) — the **ROC curve** plots sensitivity vs. (1–specificity) across all thresholds, and **AUC** (area under curve) summarizes overall discriminative ability in a single, threshold-free number (here AUC = 0.83 — good discrimination).

7.4 Logistic Regression Summary

Advantages	Disadvantages
Easy to implement	Works well only if the assumed (log-linear) dependency structure is roughly correct
Easy to interpret (sign/magnitude of β)	Performs poorly on unbalanced data (e.g. $n(1)=95$, $n(0)=5$ — the model may just ignore the rare class, which can be critical, e.g. rare disease/fraud detection)
Model reduction identical to linear regression (AIC / lasso)	—

8. Exam-Style Question Bank

These are original practice questions written in the style typically expected in a TU Dresden MS Computer Science "Digitization and Data Analytics" exam / oral exam. They are **not** a reproduction of any real, leaked past exam paper — use them for self-testing and concept-checking.

8.1 Dimension Reduction & PCA

Q1. Why do we center the data (subtract the mean) before PCA, and does the "shift" affect the principal directions?

Basic

A: PCA maximizes variance, and variance is invariant to shifts (adding a constant does not change spread). Centering only removes the (irrelevant) location so that the PCs have mean zero and the covariance matrix correctly captures the spread around the mean. The shift itself carries no information about the "shape" of the data cloud, so it does not affect the eigenvectors — only the covariance (second moment) does.

Q2. Formally state the optimization problem PCA solves for the first principal component, and explain why $\|\beta\|=1$ is required.

Core

A: Find β with $\|\beta\|=1$ such that $Y=\beta X$ maximizes $\text{var}(Y)$. The unit-norm constraint is necessary because without it, $\text{var}(\beta \cdot X)$ could be made arbitrarily large just by scaling β up — there would be no well-defined maximum. Constraining $\|\beta\|=1$ makes the problem well-posed: the solution is the eigenvector of $\text{cov}(X)$ corresponding to the largest eigenvalue, and the achieved variance equals that eigenvalue.

Q3. Prove (sketch) that the eigenvectors of the covariance matrix solve the PCA optimization problem.

Advanced

A: Maximize $\beta' \text{cov}(X) \beta$ subject to $\beta' \beta = 1$. Using a Lagrange multiplier λ : $L = \beta' \Sigma \beta - \lambda(\beta' \beta - 1)$. $\partial L / \partial \beta = 2 \Sigma \beta - 2 \lambda \beta = 0 \Rightarrow \Sigma \beta = \lambda \beta$, i.e. β must be an eigenvector of $\Sigma = \text{cov}(X)$ with eigenvalue λ . Since the objective value at the solution is $\beta' \Sigma \beta = \beta' \lambda \beta = \lambda$, to maximize we pick the eigenvector with the **largest** eigenvalue. Subsequent PCs are found by repeating under an added orthogonality constraint to previous PCs, yielding the full eigendecomposition ordered by decreasing λ .

Q4. What information does $\text{rank}(\text{cov}(X))$ give you, and when would you expect fewer than p non-zero eigenvalues?

Core

A: $\text{rank}(\text{cov}(X)) = \text{number of non-zero eigenvalues} = \text{the number of "genuinely independent" directions of variability in the data}$. It will be $< p$ whenever there is **exact** linear dependence among the p variables (e.g. one column is a linear combination of others, or $n < p$). In practice with real, noisy data this is rare exactly but eigenvalues can be numerically close to zero, motivating dimension reduction.

Q5. Explain the elbow method and the explained-variance-threshold method for choosing q . What are the risks of each?

Core

A: Elbow method: plot λ_q (scree plot) vs. q and pick q where the marginal gain drops sharply ("bend" in the curve) — subjective, can be ambiguous with a smooth decay. Threshold method: pick smallest q such that $\psi_q \geq 90\%/95\%$ — simple and reproducible, but the threshold itself is an arbitrary choice, and a high threshold can still require many PCs if variance decays slowly (e.g. near-isotropic data), defeating the purpose of dimension reduction.

Q6. What does a biplot show, and how would you read it to decide if a variable is well represented in a 2-PC solution?

Core

A: A biplot draws each original variable X_i as a vector/point with coordinates $(\text{cor}(X_i, Y_1), \text{cor}(X_i, Y_2))$. Since $\sum_{j=1}^p \rho_{ij}^2 = 1$, a variable whose arrow tip lies close to the unit circle is almost fully explained by the first two PCs; a short arrow (far inside the circle) means the variable's variance is spread over other, unshown PCs and is poorly represented in the 2D plot.

Q7. List two disadvantages of PCA and, for each, name a method that addresses it.

Basic

A: (1) Only linear dependencies — addressed by kernel PCA or autoencoders (non-linear dimension reduction). (2) Sensitive to outliers (covariance/mean are not robust statistics) — addressed by robust covariance estimators or robust PCA variants that down-weight/ignore outlying observations.

Q8. In the students'-grades example, what real-world assumption would make PCA an appropriate model, and when would it fail?

A: PCA (and the underlying single-factor "IQ" model) is appropriate if grades across subjects are driven predominantly by one (or a few) linear latent factor(s), so all subjects are strongly, linearly correlated. It fails if abilities are subject-specific and largely independent (e.g. a student excellent at math but weak at languages, uncorrelated with a global ability) — then no single (or few) linear combination(s) would explain most of the variance, and ψ_q would rise slowly, needing almost all p PCs.

8.2 t-SNE

Q9. Why does t-SNE use a Student-t (Cauchy-like) kernel in the low-dimensional space instead of a Gaussian, as used for the high-dimensional similarities?

Advanced

A: This addresses the "crowding problem": in high dimensions there is exponentially more room to place moderately-distant points than in 2D/3D, so if both P and Q used Gaussian kernels, moderately dissimilar points would be forced unnaturally close together to satisfy the probability constraints. The heavier-tailed Student-t distribution in the low-dimensional space allows moderately dissimilar points to be placed farther apart, better preserving local structure and giving visually well-separated clusters.

Q10. Why is t-SNE not directly applicable to new, unseen data points, unlike PCA?

Core

A: PCA defines an explicit, closed-form linear map $Y = \Gamma'(X - \bar{X})$ that can be applied to any new x . t-SNE has no such explicit mapping — the embedding Y is the *result* of an iterative optimization over the specific set of points used, jointly. A new point would in principle require re-running the entire optimization (or an approximate out-of-sample extension), which is computationally expensive and not part of the standard algorithm.

Q11. Two data scientists run t-SNE on the same dataset with the same hyperparameters and get visibly different plots. Is this a bug?

Applied

A: No — t-SNE initializes the low-dimensional points randomly and optimizes via stochastic gradient steps, so different random seeds (and sometimes different optimization runs) converge to different (but often qualitatively similar, e.g. same cluster memberships) embeddings. Distances *between* well-separated clusters, and even cluster sizes, are not always meaningfully comparable across runs — this is a well-known caveat when presenting t-SNE plots.

Q12. Compare PCA and t-SNE on: (a) determinism, (b) speed, (c) type of structure captured, (d) applicability to new data.

Core

A: (a) PCA deterministic (unique eigen-decomposition); t-SNE stochastic. (b) PCA fast/closed-form; t-SNE iterative and computationally intensive. (c) PCA captures global linear variance; t-SNE preserves local neighborhood structure and can capture non-linear relationships. (d) PCA trivially extends to new points via the same linear map; t-SNE does not extend to new points without re-optimization.

8.3 Autoencoders

Q13. In what precise sense is PCA a special case of an autoencoder?

Advanced

A: A linear autoencoder (no non-linear activation functions) trained to minimize squared reconstruction error, with a bottleneck of size q , learns a bottleneck subspace that spans the same q -dimensional subspace as the first q principal components of PCA (the specific basis found may differ by rotation, but the *subspace*, and hence the achievable reconstruction error, coincides). Non-linear activations in the encoder/decoder are what let autoencoders go beyond PCA and capture non-linear manifolds.

Q14. Why can an autoencoder be applied to new, out-of-sample data while t-SNE cannot?

Core

A: The autoencoder's encoder is an explicit, learned parametric function (a neural network with fixed weights after training) mapping any input x to its code — you simply run a new x through the (already-trained) encoder. t-SNE has no such standalone function; its embedding is defined only jointly over the specific training set via iterative optimization, with no natural "predict" step for new points.

Q15. Give one practical risk of using an autoencoder trained on one dataset to encode data from a different, related domain.

A: Autoencoders only reconstruct well data that resembles their training distribution; feeding in out-of-distribution data can produce a poor/misleading code (and poor reconstruction), since the network has learned features specific to the training data's manifold, not a general-purpose compression scheme.

8.4 Linear Regression

Q16. Derive the OLS estimator $\hat{\beta} = (X'X)^{-1}X'Y$ from the least-squares objective.

Core

A: Minimize $S(\beta) = \|y - X\beta\|^2 = (y - X\beta)'(y - X\beta)$. Expand: $S(\beta) = y'y - 2\beta'X'y + \beta'X'X\beta$. $\partial S / \partial \beta = -2X'y + 2X'X\beta = 0 \Rightarrow X'X\beta = X'y \Rightarrow \beta = (X'X)^{-1}X'y$ (assuming $X'X$ invertible, i.e. X has full column rank).

Q17. Under what two circumstances does computing $(X'X)^{-1}$ become problematic, and what are the standard remedies?

Core

A: (1) X is very large (high p , e.g. $p > n$ or close to it) — inversion is expensive/numerically unstable; remedy: dimension reduction (PCR) or regularization (ridge/lasso) which replace the exact inverse with a stabilized or lower-dimensional computation. (2) Columns of X are (nearly) linearly dependent — multicollinearity makes $X'X$ (nearly) singular, inflating $\text{Var}(\beta)$; remedy: drop/combine correlated variables, use PCR, or use lasso/ridge which shrink and can drop redundant coefficients.

Q18. Why can plain R^2 never be used to fairly compare a full model against a reduced (nested) sub-model?

Core

A: Adding any variable, even pure noise uncorrelated with y , can only keep SS_{residual} the same or decrease it (OLS always weakly improves in-sample fit with more regressors), so R^2 is non-decreasing as variables are added. This means R^2 will always (weakly) favor the larger model regardless of whether the new variables carry real explanatory power — hence penalized criteria (adjusted R^2 , AIC) that explicitly charge a cost per added parameter are needed for fair comparison.

Q19. A student log-transforms Y in a regression to "improve" R^2 . What issues could this cause?

Applied

A: Transforming Y changes what $SS_{\text{total}}/SS_{\text{residual}}$ (and hence R^2/AIC) are actually measuring, so the resulting R^2 or AIC is **not comparable** to models fit on the untransformed y — you'd be comparing apples to oranges. It also changes the error assumptions (homoscedastic, normal errors on $\log(y)$ is a different assumption than on y) and complicates prediction, since predictions need to be back-transformed (naively exponentiating a linear predictor on log-scale introduces bias — a Jensen's-inequality-type issue since $E[\exp(Z)] \neq \exp(E[Z])$).

Q20. Compare backward AIC-based selection, PCR, and (adaptive) lasso as model reduction strategies.

Core

A: Backward/AIC: interpretable (keeps original variables), but requires fitting many nested sub-models (worst case $p(p+1)/2$), and is a discrete/greedy search that can miss the globally optimal subset. PCR: fast, handles multicollinearity well, but loses interpretability (coefficients are on PCs, not original variables) and still requires measuring/collecting every original variable. Lasso: single optimization with continuous hyperparameter λ (tunable via cross-validation), performs continuous shrinkage plus automatic exact variable selection, typically faster and often sparser than backward selection; adaptive lasso improves selection consistency by using a data-dependent penalty per coefficient.

Q21. In the iris petal-length regression, why do the two species dummy coefficients (versicolor=1.46, virginica=1.97) both being positive and significant matter for interpretation?

Applied

A: The dummies are relative to the omitted baseline category (setosa). Positive, significant coefficients mean that, holding Sepal.Length, Sepal.Width and Petal.Width fixed, both versicolor and virginica have systematically **larger** Petal.Length than setosa, with virginica's effect being larger still than versicolor's ($1.97 > 1.46$) — consistent with the well-known visual separation of the three species by petal size.

Q22. What assumptions of linear regression should be checked *after* fitting, and what diagnostic plot(s) would you use for each?

A: Normality of residuals — Q-Q plot of residuals. Homoscedasticity (constant variance) — residuals vs. fitted values plot (should show no funnel/pattern). No/low multicollinearity — correlation matrix or variance inflation factors (VIF) among predictors. Linearity — residuals vs. each predictor (no systematic curvature). Independence of errors — relevant especially for time-ordered or spatially-correlated data (e.g. Durbin-Watson test).

8.5 Logistic Regression

Q23. Why can't we just apply OLS directly to a 0/1 target instead of using logistic regression?

Basic

A: A linear probability model (OLS on 0/1 y) can predict probabilities outside $[0,1]$, violating the basic requirement that a probability lies in $[0,1]$; it also violates the constant-variance assumption (the variance of a Bernoulli variable, $p(1-p)$, depends on p, which itself depends on x, i.e. errors are heteroscedastic by construction). Logistic regression's sigmoid link function keeps predictions in $(0,1)$ and models the log-odds linearly instead, which is the mathematically consistent way to model a probability as a function of covariates.

Q24. Derive the sigmoid function starting from the log-odds (logit) model.

Core

A: Assume $\log[P(Y=1|x)/P(Y=0|x)] = x'\beta$. Since $P(Y=0|x) = 1 - P(Y=1|x)$, let $p = P(Y=1|x)$: $\log[p/(1-p)] = x'\beta \Rightarrow p/(1-p) = \exp(x'\beta) \Rightarrow p = \exp(x'\beta)/(1-p) \Rightarrow p + p \cdot \exp(x'\beta) = \exp(x'\beta) \Rightarrow p = \exp(x'\beta)/(1 + \exp(x'\beta)) = 1/(1 + \exp(-x'\beta)) = \sigma(x'\beta)$.

Q25. Interpret $\hat{\beta}_j$ in a logistic regression model (in terms of odds), and why is this interpretation more nuanced than in linear regression.

Advanced

A: Holding other variables fixed, $\exp(\beta_j)$ is the multiplicative change in the **odds** $P(Y=1)/P(Y=0)$ for a one-unit increase in x_j — e.g. $\exp(\beta_j) = 1.5$ means the odds increase by 50%. This is more nuanced than linear regression's direct "additive change in y" interpretation because the effect on the actual **probability** $P(Y=1)$ (not the odds) is non-linear and depends on the current value of $x'\beta$ (the marginal effect on probability, $\sigma(z)(1-\sigma(z))\beta_j$, is largest near $p=0.5$ and shrinks near 0 or 1).

Q26. Explain sensitivity and specificity, and why a single accuracy number can be misleading for imbalanced classes.

Core

A: Sensitivity (true positive rate) = $TP/(TP+FN)$ — the fraction of actual positives correctly found. Specificity = $TN/(TN+FP)$ — the fraction of actual negatives correctly found. With imbalanced classes (e.g. 95 negatives, 5 positives), a trivial classifier predicting "always negative" achieves 95% accuracy while having 0% sensitivity — completely failing to detect the (possibly critical) positive class. Hence sensitivity/specificity (or precision/recall, F1, AUC) must be reported alongside accuracy.

Q27. What does the ROC curve plot, and what does AUC=0.5 vs. AUC=1.0 mean?

Core

A: The ROC curve plots sensitivity (TPR) against $1 - \text{specificity}$ (FPR) as the classification threshold is varied from 1 down to 0. AUC=0.5 corresponds to the diagonal line — performance indistinguishable from random guessing. AUC=1.0 means perfect separation of the two classes at some threshold (a curve that goes straight up to $(0,1)$ then right to $(1,1)$). AUC is threshold-independent and thus useful for comparing classifiers overall, not just at one operating point.

Q28. Why might lowering the classification threshold from 0.5 be a reasonable business/clinical decision even though it decreases accuracy? Applied

A: Lowering the threshold increases sensitivity (fewer false negatives) at the cost of specificity (more false positives). In settings where missing a true positive is far more costly than a false alarm (e.g. disease screening, fraud detection), this trade-off is worthwhile even if raw accuracy drops slightly, because the asymmetric cost of errors — not accuracy alone — should drive the threshold choice.

Q29. How is model reduction (variable selection) in logistic regression related to what you do in linear regression? Core

A: Identical toolbox — significance testing (p -values on β_j), AIC-based backward elimination, and (adaptive) lasso all directly generalize to logistic regression, replacing OLS's squared error with the model's (negative log-)likelihood as the fit criterion being penalized/compared, e.g. lasso-penalized logistic regression maximizes the log-likelihood minus $\lambda \|\beta\|_1$.

Q30. What's the difference between binary logistic regression and multinomial regression, and when do you need the latter? Core

A: Binary logistic regression models $P(Y=1|x)$ for a two-class target. Multinomial regression is its extension for $K>2$ unordered classes, modeling $P(Y=k|x)$ for each class k simultaneously (typically via a softmax generalization of the sigmoid, with one reference/baseline category). You need it whenever the target has more than two unordered categories, e.g. predicting one of the three iris species directly rather than one-vs-rest.

8.6 Cross-Topic / Synthesis Questions

Q31. You have 200 highly correlated predictors and want to run a regression. Propose and justify a full pipeline using concepts from this lecture. Synthesis

A: (1) Diagnose multicollinearity (correlation matrix / VIF); since predictors are highly correlated, $(X'X)^{-1}$ will be poorly conditioned. (2) Either (a) apply PCA/PCR: extract PCs explaining e.g. 90–95% variance (scree plot/threshold), regress y on those PCs — fast, stable, but coefficients not directly interpretable in original variable terms; or (b) apply (adaptive) lasso directly on the original 200 variables — retains interpretability of surviving coefficients and gives automatic sparse variable selection, tuning λ via cross-validation. (3) Evaluate the chosen model via R^2 /adjusted R^2 (or AIC) and residual diagnostics (normality, homoscedasticity). Lasso is generally preferred when interpretability of individual original features matters; PCR is preferred when you mainly need predictive stability and don't need original-variable coefficients.

Q32. Both PCA and logistic-regression AIC-based backward selection are "reduce the dimension without losing much information" procedures. What is fundamentally different about what each is trying to preserve? Synthesis

A: PCA is **unsupervised**: it preserves as much variance in X **itself** as possible, completely ignoring any target Y . Backward/AIC-based selection in regression is **supervised**: it keeps exactly those X -variables that best explain/predict a specific target Y , potentially discarding variables with huge variance in X but no relationship to Y , and potentially keeping a low-variance variable in X that happens to be highly predictive of Y . This is why PCR (PCA first, then regress on top PCs) can sometimes underperform lasso: the PCs capturing the most variance in X are not guaranteed to be the ones most correlated with Y .

Q33. Why does the lecture describe the autoencoder, and not PCA or t-SNE, as being usable for "out-of-sample" data? Give the general principle. Synthesis

A: General principle: a method is usable out-of-sample if and only if it produces an explicit, reusable mapping $f: X \rightarrow Y$ that does not depend on re-solving a joint problem over the whole dataset for each new point. PCA does have such a closed-form map ($Y = \Gamma'(X - \bar{X})$) so it is *also* usable out-of-sample (the lecture's specific "advantage" framing for autoencoders should not be read as PCA lacking this property) — but autoencoders extend this to **non-linear** maps, learned via the encoder network, whereas t-SNE has no explicit map of any kind (linear or non-linear) — its "solution" is the specific set of low-dimensional coordinates jointly optimized for the training points, with no function to apply to new x .

Q34. Sketch how you would evaluate whether logistic regression or a decision tree (from Lecture 6) is the better classifier on a given dataset. Synthesis

A: Fit both on a training split; evaluate both on a held-out test set with the same metrics (confusion matrix, accuracy, sensitivity, specificity, ROC/AUC). Compare interpretability requirements (logistic regression gives global linear log-odds coefficients; decision trees give explicit if/then rules that may be easier for domain experts to audit but can overfit without pruning). Check whether the class-probability boundary is likely to be linear-in-logit (favors logistic regression) or has strong interactions/non-linear thresholds (may favor trees). Also compare robustness on unbalanced classes and whether calibrated probabilities (not just class labels) are needed downstream — logistic regression outputs are directly interpretable probabilities, tree leaf proportions less reliably so.

9. Research-Oriented Questions

These go beyond the slide content, connecting the lecture to current methodology and open questions — useful for oral exams, seminar discussions, or thesis-proposal style reasoning.

R1. PCA assumes the "interesting" structure is the direction of maximum variance. When can this assumption actively mislead an analysis? Research

A: Whenever the class-discriminative or scientifically interesting direction is **not** the direction of maximum variance — e.g. two classes separated along a low-variance axis while a nuisance factor (illumination in images, batch effects in genomics) dominates the variance. In such cases PCA's top components can be dominated by the nuisance factor, discarding exactly the signal of interest. This motivates supervised or semi-supervised alternatives (e.g. LDA, partial least squares, contrastive representation learning) that explicitly use label information when it exists.

R2. Modern non-linear dimension reduction methods (UMAP, and t-SNE successors) are widely used but their global geometry (relative cluster distances/sizes) is often not meaningful. What does this imply for using such plots as scientific evidence?

A: Because both t-SNE and UMAP prioritize preserving local neighborhoods over global distances, visual features like "cluster A is twice as far from B as from C" or "cluster A is bigger than cluster B" are frequently artifacts of the optimization/hyperparameters (perplexity, learning rate, initialization), not real properties of the data. Best practice in current research is to treat these embeddings as hypothesis-generating visualizations, and validate any apparent structure (e.g., cluster separation) using an independent, more principled measure (e.g. a supervised classifier's accuracy, a statistical test on cluster assignments, or the original high-dimensional distances) before drawing scientific conclusions.

R3. Autoencoders are described here as (dense) neural networks. How do variational autoencoders (VAEs) change the objective, and why does this matter for generative modeling versus pure compression? Research

A: A plain autoencoder only minimizes reconstruction error, with no constraint on the geometry of the latent code — nearby codes need not decode to similar/plausible outputs, and there's no way to sample new, realistic data from the latent space. A VAE additionally regularizes the latent distribution toward a known prior (typically a standard Gaussian) via a KL-divergence term added to the loss, making the latent space smooth and samplable. This upgrades the model from pure lossy compression (this lecture's framing) to a generative model capable of producing new synthetic data — at some cost to reconstruction sharpness (the classic reconstruction-vs-regularization trade-off, tuned via a β -VAE-style weighting).

R4. The lecture recommends (adaptive) lasso for model reduction. What theoretical property does the "adaptive" weighting give you that plain lasso lacks? Research

A: Plain lasso is known (Zou, 2006) to not, in general, possess the "oracle property" — even asymptotically, its variable-selection can be inconsistent for certain designs, and it can over-shrink large true coefficients due to a uniform penalty. Adaptive lasso reweights the L1 penalty per coefficient using weights derived from a preliminary (e.g. OLS or ridge) estimate, penalizing coefficients believed to be small more heavily and truly-large coefficients less. Under suitable regularity conditions, this restores the oracle property: asymptotically correct variable selection combined with the same asymptotic distribution as if the true (sparse) model were known in advance.

R5. Logistic regression is a special case of the exponential family / GLM framework. What is gained by recognizing this, especially regarding "goodness of fit"? Research

A: Recognizing logistic regression as a Generalized Linear Model (Bernoulli family, logit link) unlocks the shared GLM machinery: maximum-likelihood estimation via iteratively reweighted least squares, deviance as a likelihood-based analogue of SS_{residual} (enabling likelihood-ratio tests between nested models, analogous to comparing R^2/AIC in OLS), and a principled way to swap in other links/families (Poisson regression for counts, multinomial/softmax for K-class problems) while reusing the same inferential toolbox (Wald tests, AIC, deviance-based goodness-of-fit) covered in this lecture for the Gaussian linear-model case.

R6. The lecture notes logistic regression "works poorly for unbalanced data." What are two distinct modern remedies, and what trade-off does each involve? Research

A: (1) Resampling (oversampling the minority class, e.g. SMOTE, or undersampling the majority) — rebalances the training distribution but can introduce synthetic-sample artifacts (SMOTE) or discard real information (undersampling), and changes what the fitted probabilities actually estimate unless corrected. (2) Cost-sensitive learning / class-weighted likelihood — directly weight the log-likelihood contributions of the minority class more heavily during estimation, preserving the true data distribution (so probabilities remain calibrated) but requiring the analyst to choose/justify the cost ratio, which is often domain-specific (e.g. cost of missing a fraud case vs. a false alarm) and not always known precisely.

R7. How would you extend the PCA framework in this lecture to streaming / online data, where the full covariance matrix cannot be stored or recomputed each time? Research

A: Standard batch PCA needs the full $n \times p$ data (or its $p \times p$ covariance matrix) in memory to eigen-decompose. For streaming data, incremental/online PCA methods update an approximate eigen-basis using only the new observation and the current estimate (e.g. via rank-one covariance updates, stochastic gradient ascent on the variance objective, or Oja's rule), trading exact optimality for constant per-sample memory/compute and the ability to adapt to slowly drifting data distributions (concept drift) — an active research area at the intersection of this lecture's PCA and streaming/online learning.

R8. Could the "linear regression → logistic regression" progression in this lecture be unified into a single estimation framework, and what would that buy you pedagogically and practically?

Research

A: Yes — both are special cases of Generalized Linear Models: $y|x$ from an exponential-family distribution with mean linked to $x'\beta$ via a link function (identity link + Gaussian noise = linear regression; logit link + Bernoulli = logistic regression). Pedagogically, this unifies OLS's "minimize squared error" and logistic regression's "maximize Bernoulli likelihood" as instances of one maximum-likelihood estimation principle. Practically, it means the same model-reduction tools (AIC, lasso-penalized likelihood, backward selection) transfer directly across response types, and lets an analyst swap response distributions (Poisson counts, Gamma for positive skewed data, multinomial for multi-class) without re-deriving estimation theory from scratch.

10. One-Page Cheat Sheet

Method	Key formula	Type	Main limitation
PCA	$Y = \Gamma'(X - \bar{X}), \text{cov}(X) = \Gamma \Lambda \Gamma'$	Unsupervised, linear	Linear structure only, outlier-sensitive
t-SNE	$\min D_{KL}(P \ Q)$	Unsupervised, non-linear, local	Stochastic, no out-of-sample map
Autoencoder	$\min \ X - \text{decoder}(\text{encoder}(X))\ ^2$	Unsupervised, non-linear (NN)	Needs data similar to training set
Linear regression	$\beta = (X'X)^{-1}X'Y$	Supervised, continuous Y	Linear-in-parameters, sensitive to collinearity/outliers
Logistic regression	$P(Y=1 x) = 1/(1 + e^{-x'\beta})$	Supervised, categorical Y	Poor on unbalanced classes

Model-reduction techniques (shared by linear & logistic regression)

Technique	Idea	Trade-off
Backward / AIC	Iteratively drop the least useful variable	Interpretable, but many sub-models to fit
PCR	Regress on top PCs of X instead of X itself	Stable, handles collinearity; coefficients hard to interpret
(Adaptive) Lasso	$\ y - X\beta\ ^2 + \lambda \ \beta\ _1 \rightarrow \min$	Fast, sparse, tunable λ ; adaptive version restores oracle property

Goodness-of-fit quick reference

Regression	Classification
R^2 , adjusted $\hat{R}^2$, AIC	Accuracy, sensitivity, specificity, ROC/AUC, confusion matrix
p-values on β_j (assumes $\varepsilon \sim N(0, \sigma^2 I)$)	p-values on β_j (Wald tests from MLE)

Final exam tip: When a question gives you a table of β and p-values (as in the iris examples), always (1) read the sign of β for direction of effect, (2) compare $|\beta|$ only if variables are on comparable scales, (3) check p-value against 0.05 for significance, (4) connect to R^2 /AIC or accuracy/AUC for overall model quality, and (5) mention whether model reduction is warranted.